# Vorlesung: Grundlagen der Computergrafik

Kapitel 2: 2D-Computergrafik

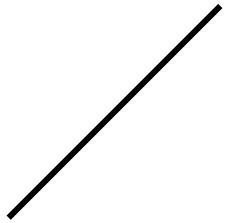
## Kapitel 2.3:

# SVG
# Scalable Vector Graphics

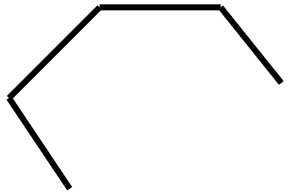
Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier



# SVG: Grafische Elemente



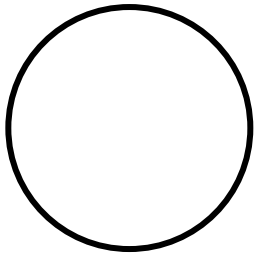
line



polyline



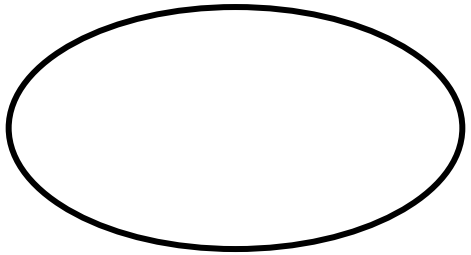
polygon



circle



rect



ellipse

Beispiele: https://www.st.uni-trier.de/lectures/S26/CG/svg/

User:
Password:

# Beispiel: Linien

linien.svg

```
<?xml version="1.0"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
  "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

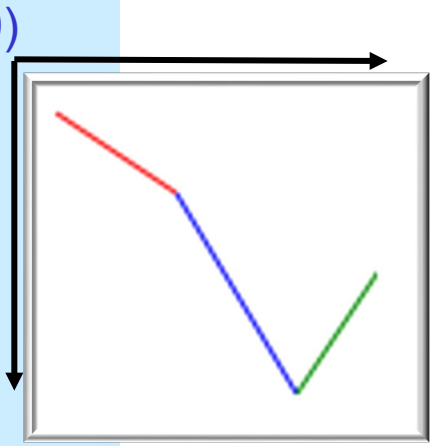
<svg xmlns="http://www.w3.org/2000/svg"
     xmlns:xlink="http://www.w3.org/1999/xlink"
     version="1.1"
     baseProfile="full"
     width="100" height="120">

  <line stroke="red" x1="20" y1="30" x2="50" y2="50"/>

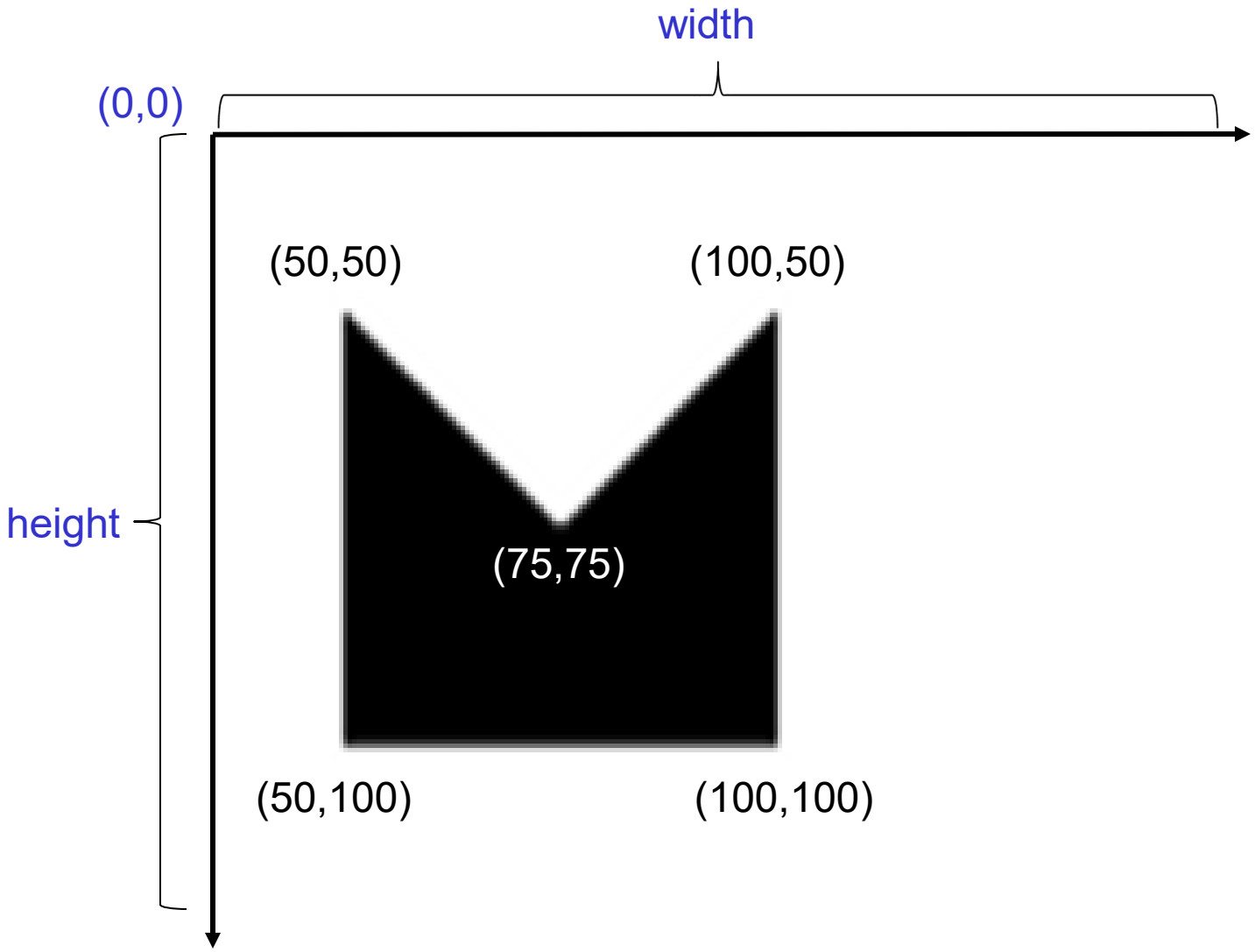
  <line stroke="blue" x1="50" y1="50" x2="80" y2="100"/>

  <line stroke="green" x1="80" y1="100" x2="120" y2="40"/>

</svg>
```



# Koordinatensystem



# SVG: Polygone

krone.svg

```
<?xml version="1.0"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
  "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

<svg xmlns="http://www.w3.org/2000/svg"
     xmlns:xlink="http://www.w3.org/1999/xlink"
     version="1.1"
     baseProfile="full"
     width="100" height="120" >

<polygon points="50,50,  75,75, 100,50, 100,100, 50,100"/>

</svg>
```

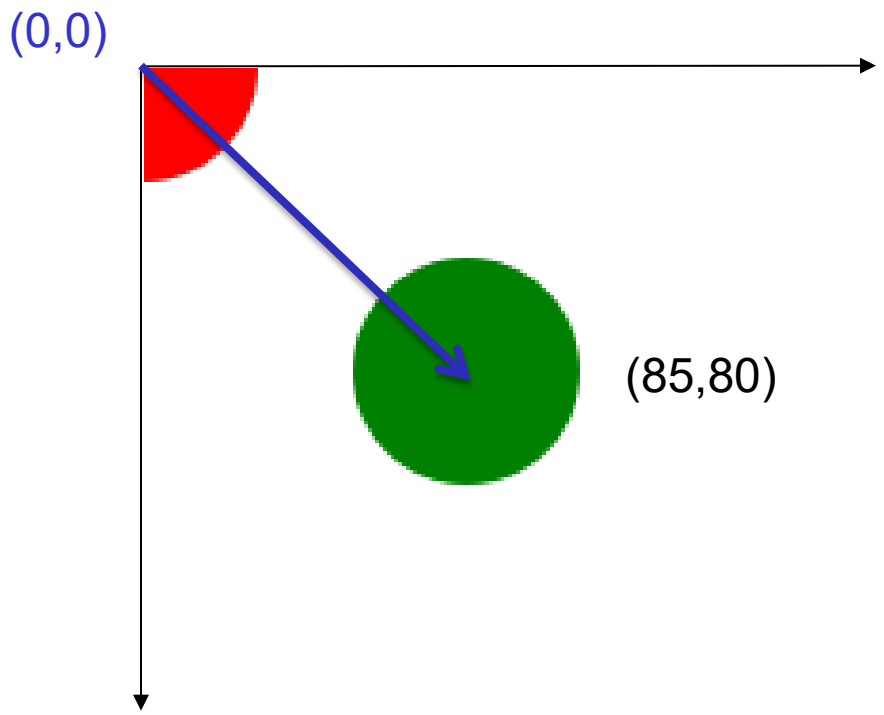


# SVG: Transformation

kreisUrsprung.svg

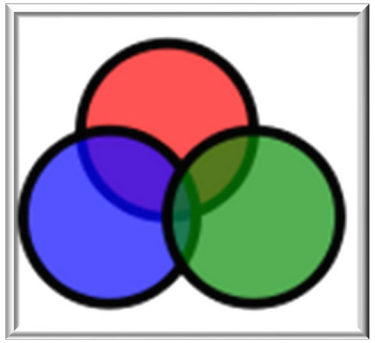
```
<circle r="30" fill="red"/>

<circle r="30" fill="green" transform="translate(85,80)" />
```



# SVG: Gruppierung

kreise.svg



```
<?xml version="1.0"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
  "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

<svg xmlns="http://www.w3.org/2000/svg"
     xmlns:xlink="http://www.w3.org/1999/xlink"
     version="1.1"
     baseProfile="full"
     width="120" height="120">

 <g fill-opacity="0.7" stroke="black" stroke-width="2">
   <circle r="30" fill="red"   transform="translate(55,50)"/>
   <circle r="30" fill="blue"  transform="translate(35,80)"/>
   <circle r="30" fill="green" transform="translate(85,80)"/>
 </g>

</svg>
```

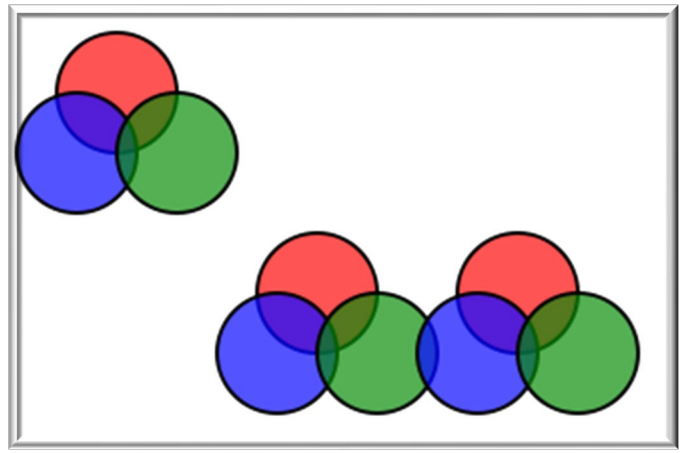
Gruppierung

# SVG: Wiederverwendung

```
<g id="kreise" fill-opacity="0.7" stroke="black" stroke-width="2">
   <circle r="30" fill="red"   transform="translate(55,50)"/>
   <circle r="30" fill="blue"  transform="translate(35,80)"/>
   <circle r="30" fill="green" transform="translate(85,80)"/>
</g>

<use xlink:href="#kreise" transform="translate(100,100)" />

<use xlink:href="#kreise" transform="translate(200,100)" />
```

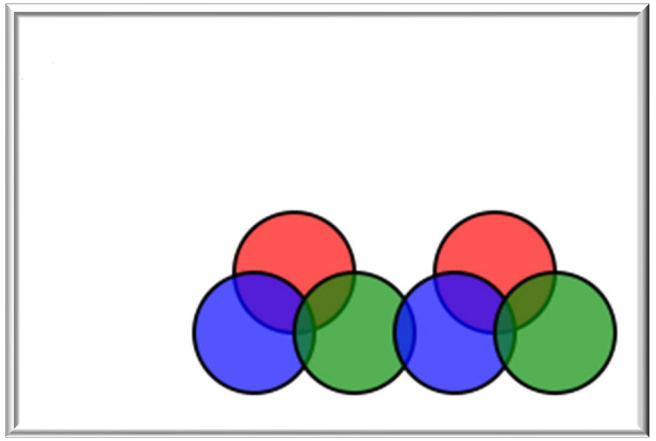


# SVG: Wiederverwendung

```
<defs>
  <g id="kreise" fill-opacity="0.7" stroke="black" stroke-width="2">
    <circle r="30" fill="red"     transform="translate(55,50)" />
    <circle r="30" fill="blue"    transform="translate(35,80)" />
    <circle r="30" fill="green"   transform="translate(85,80)" />
  </g>
</defs>

<use xlink:href="#kreise"  transform="translate(100,100)" />

<use xlink:href="#kreise"  transform="translate(200,100)" />
```



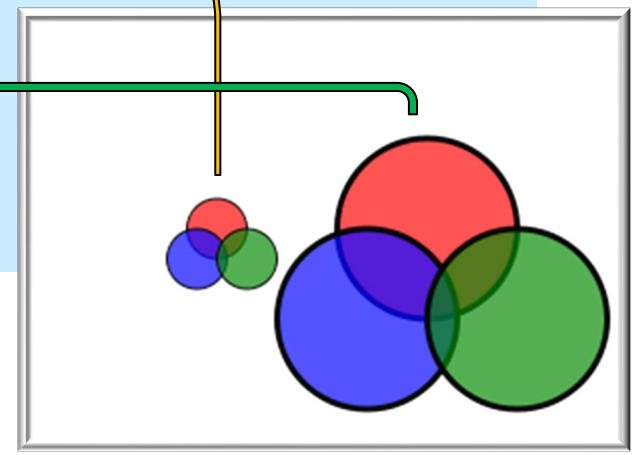
# SVG: Skalieren

```
<defs>
  <g id="kreise" fill-opacity="0.7" stroke="black" stroke-width="2">
    <circle r="30" fill="red"     transform="translate(55,50)" />
    <circle r="30" fill="blue"    transform="translate(35,80)" />
    <circle r="30" fill="green"   transform="translate(85,80)" />
  </g>
</defs>

 <use xlink:href="#kreise"
          transform="translate(80,100) scale(0.5)"/>

 <use xlink:href="#kreise"
          transform="translate(130,50) scale(1.5)"/>

</svg>
```



# SVG: Pfade

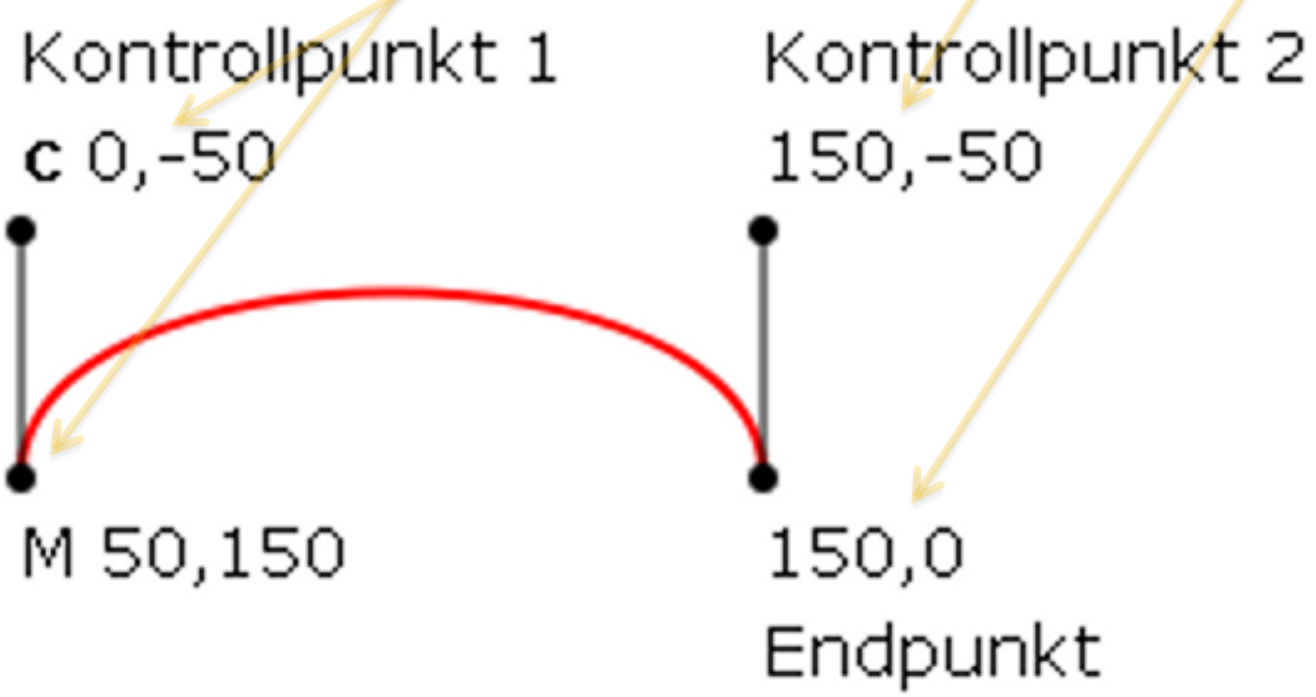
| | |
|---|---|
| **M** | Stift positionieren ohne Zeichnen |
| **L** | Linie zur angegebenen Positions |
| **H** | Horizontale Linie |
| **V** | Vertikale Linie |
| **C** | Stark gekrümmte kubische Bezierkurve |
| **S** | Schwach gekrümmte kubische Bezierkurve |
| **Q** | Stark gekrümmte quadratische Bezierkurve |
| **T** | Schwach gekrümmte quadratische Bezierkurve |
| **A** | Elliptischer Bogen |
| **Z** | Geschlossener Pfad (Rücksprung) |

*Großbuchstaben enthalten absolute Positionen:* **M 50,150**

*Kleinbuchstaben definieren relative Positionen:* **m 50,150**

# SVG: Pfade

```
<path d="M 50,150 c 0,-50 150,-50 150,0" />
```



# SVG: Pfade

pfad.svg

```
<?xml version="1.0"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"  "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

<svg xmlns=http://www.w3.org/2000/svg  xmlns:xlink="http://www.w3.org/1999/xlink"
     version="1.1"      baseProfile="full" width="100" height="120">

<path d="M50,50
         C55,50 75,70 75,75
         C75,70 90,50 100,50
         L100,100
         L50,100Z"

      style="stroke:red;  stroke-width: 4; fill: none"/>

</svg>
```



CSS kann verwendet werden, um Attributwerte zu setzen. Dies geht sowohl über das Attribut **`style`**, das Element **`<style>`** oder mittels einer externen CSS-Datei.

# SVG: Text

text.svg




```
<defs>
   <text id="txt" font-size="16" font-family="Arial"
         font-weight="bold">Scalable Vector Graphics</text>
</defs>

<use xlink:href="#txt" x="10" y="30"/>
<use xlink:href="#txt" x="10" y="50"/>
<use xlink:href="#txt" x="10" y="70"/>
<use xlink:href="#txt" x="10" y="70"  transform="translate(0,4)" />
<use xlink:href="#txt" x="10" y="100" transform="rotate(45,10,100)" stroke="red" />
<use xlink:href="#txt" x="10" y="100" transform="rotate(65,10,100)" stroke="red" />
<use xlink:href="#txt" x="10" y="100" transform="rotate(85,10,100)" stroke="red" />
```

**rotate(30)** - positive Winkel drehen das Objekt im Uhrzeigersinn,
**rotate(-130)** - negative Winkel drehen das Objekt gegen den Uhrzeigersinn.
In beiden Fällen wird Nullpunkt des temporären Koordinatensystems als Drehpunkt verwendet.
**rotate(45,10,100)** - Drehpunkt ist die Koordinate 10,100.

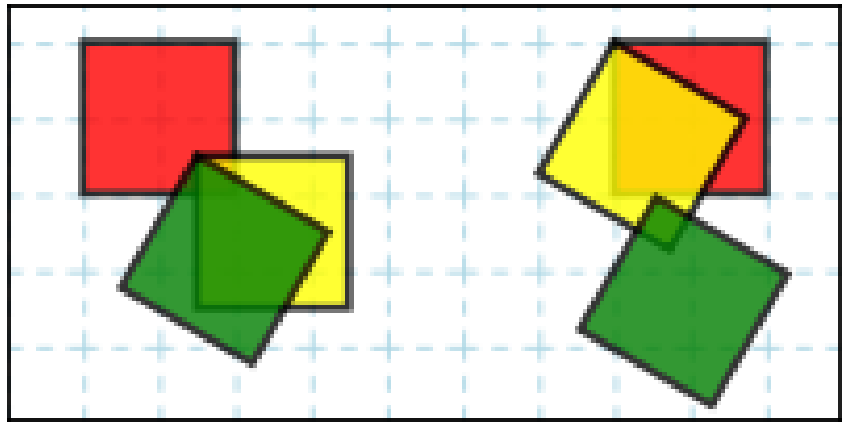
nestedTransformation.svg

# Verschachtelte Transformationen

```
<use xlink:href="#example" fill="red" />
<g transform="translate(15, 15)“ fill="yellow">
    <use xlink:href="#example" />
    <g transform="rotate(30)" fill="green">
        <use xlink:href="#example" />
    </g>
</g>
```

```
<use xlink:href="#example" fill="red" />
<g transform="rotate(30)" fill="yellow">
    <use xlink:href="#example" />
    <g transform="translate(15, 15)" fill="green">
        <use xlink:href="#example" />
    </g>
</g>
```

$T * R * p$



$R * T * p$

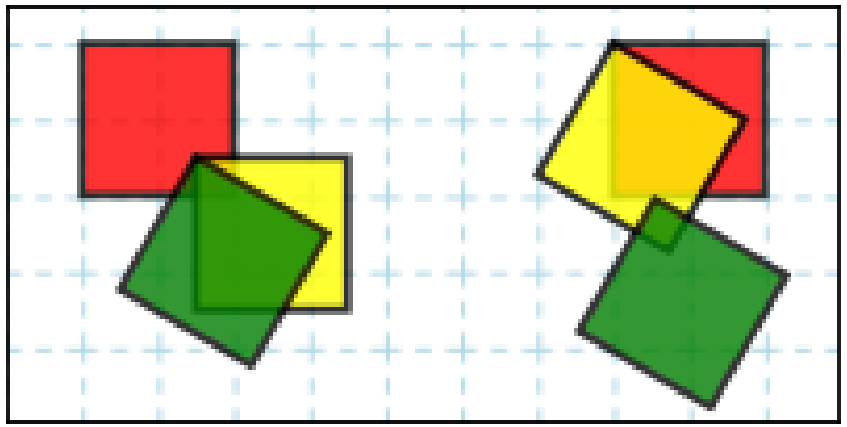
nestedTransformation2.svg

# Nacheinander ausgeführte Transformationen

```
<use xlink:href="#example" fill="red" />
<use xlink:href="#example“
     transform="translate(15,15)“ fill="yellow"/>
<use xlink:href="#example“
     transform="translate(15,15) rotate(30)"
     fill="green"/>
```

```
<use xlink:href="#example" fill="red" />
<use xlink:href="#example“
     transform="rotate(30)" fill="yellow"/>
<use xlink:href="#example“
     transform="rotate(30) translate(15, 15)"
     fill="green"/>
```

$$T * R * p$$



$$R * T * p$$

rotationsmatrix.svg

# Transformationsmatrix in SVG

## Translation:

$$\mathrm{matrix}(a,b,c,d,e,f) = \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix}$$

```
<line stroke="red" stroke-width="10"
      x1="0" y1="0"
      x2="80" y2="80"
      transform="matrix(0.707, 0.707, -0.707, 0.707, 0, 0)" />
```

Drehung um 45°

$$R = \begin{pmatrix} 0.707 & -0.707 & 0 \\ 0.707 & 0.707 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Perspektivische Projektion

$$\begin{pmatrix} d & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & d \end{pmatrix}$$

http://editsvgcode.com

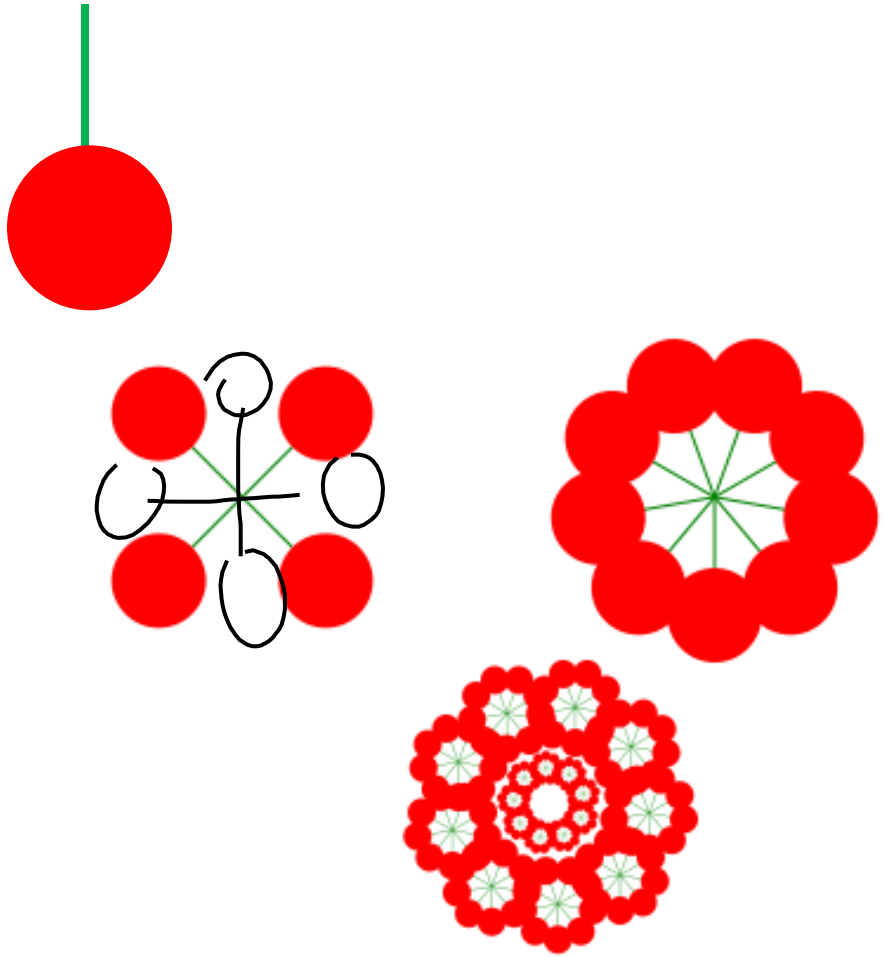
https://www.st.uni-trier.de/lectures/S26/CG/svg

**https://www.st.uni-trier.de/lectures/S26/CG/svg/aufgabe.svg**

# Aufgabe

aufgabe.svg

```
<defs>
 <g id="g1">
   <line x1="0" y1="0" x2="0" y2="40" stroke=“green"/>
   <circle r="20" fill="red" cx="0" cy="40"/>
 </g>
</defs>
```



- Gruppieren und transformieren Sie geschickt, um ähnliche Bilder mit SVG zu erzeugen.

Zum Nachschlagen:
https://wiki.selfhtml.org/wiki/SVG

# 2D Computergrafik



## Animationen mit SVG

Zum Nachschlagen: https://wiki.selfhtml.org/wiki/SVG

# SVG: Animation der Farbe [anim1.svg]

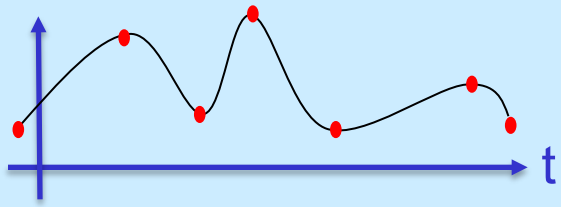
```
<?xml version="1.0"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
  "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

<svg xmlns="http://www.w3.org/2000/svg"    xmlns:xlink="http://www.w3.org/1999/xlink"
     version="1.1"      baseProfile="full"     width="520" height="520" viewport="0 0 520 520">

 <circle r="50" cx="100" cy="100" fill="red">
    <animate attributeName="fill" attributeType="CSS"
                    from="lime" to="red" begin="2s" dur="4s" fill="freeze"/>
 </circle>


<circle r="50" cx="200" cy="100" fill="red">
    <animate attributeName="fill" attributeType="CSS"
                    begin="0s" dur="2s" repeatCount="indefinite"
                    values="red;green;blue;black;yellow"/>
 </circle>
</svg>
```

Letzter Zustand bleibt nach Animationsende erhalten.



# SVG: Animierte Position

anim2.svg

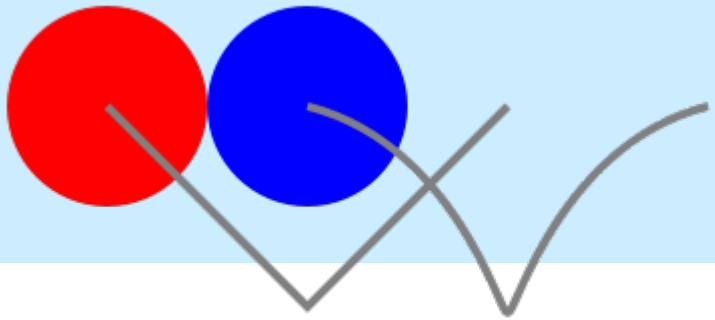
```
<?xml version="1.0"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
  "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">

<svg xmlns="http://www.w3.org/2000/svg"    xmlns:xlink="http://www.w3.org/1999/xlink"
     version="1.1"      baseProfile="full"    width="520" height="520" viewport="0 0 520 520">


 <circle r="50" cx="100" cy="100" fill="red">
     <animateMotion path="M 0 0 L 100 100 L 200 0"
                                  dur="5s" fill="freeze"/>
 </circle>

<circle r="50" cx="200" cy="100" fill="blue">
    <animateMotion
         path="M 0 0 C80,20 100,120 100,100 C100,120 120,20 200,0"
         dur="5s" fill="freeze"/>
 </circle>

</svg>
```



# Aufgabe

groups.svg

- Animieren Sie die Blumen
  - Drehen
  - Wachsen
  - (Ver-)Blühen
  - …



rotate
scale
translate

```
<animateTransform   attributeName="transform"
                    attributeType="XML" type="rotate"
                    from="1,300,400" to="360,300,400"
                    additive="sum" begin="1s" dur="5s"
                    fill="freeze" repeatCount="indefinite"/>
```

Tipp zum Nachschlagen: https://wiki.selfhtml.org/wiki/SVG